

SCHEME VALIDATORS : A GUIDE TO LEARN PRAHARI

1.INTRODUCTION

2.KNOWLEDGE

2.1 EXCEPTIONS

An exception is the way python says “something went wrong here”. If not used the whole code stops functioning and crashes with a traceback. It is responsible to handle runtime errors and improve program readability by ensuring the program doesn't terminate in the middle of something.

The basic format of exception in python is as follows:

```
try:
    # Code
except SomeException:
    # Code
else:
    # Code
Finally:
    # Code
```

Not only that but we can create classes that inherit exceptions as:

```
class ConnectorError(Exception): ...
class
TransientFetchError(ConnectorError): ...
# safe to retry
class
PermanentFetchError(ConnectorError): ...
# don't retry
```

When you build applications, standard errors like ValueError or TypeError are often too vague. Creating your own error classes allows you to bundle specific

debugging details—such as error codes, timestamps, or failed user inputs—directly into the exception object.

2.2 DATACLASS

In Python, **dataclasses** (introduced in Python 3.7 via the `dataclass` module) provide a clean, automated way to generate special methods—like `__init__()`, `__repr__()`, and `__eq__()` for classes that primarily exist to store data.

Instead of writing boilerplate code to assign attributes during initialization, you declare typed fields at the class level, and Python handles the constructor logic for you.

With dataclass,

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Book:
```

```
    title: str
```

```
    author: str
```

```
    pages: int
```

```
    price: float = 0.0
```

USAGE

```
b1 = Book("The Hobbit", "J.R.R. Tolkien",
310, 14.99)
```

```
b2 = Book("The Hobbit", "J.R.R. Tolkien",
310, 14.99)
```

```
print(b1)
```

```
print(b1 == b2)
```

2.3 GENERICS

In Python, **Generics** (introduced via type hinting in PEP 484 and simplified in Python 3.12) allow you to define classes, functions, and data structures that work across multiple types while maintaining strict type safety.

Without generics, container types or functions either force you to use **Any** (losing type checking) or duplicate code for each type.

Level 1 👍

```
from typing import TypeVar
```

```
T = TypeVar("T")
```

```
def get_first(items: list[T]) -> T:
```

```
    return items[0]
```

Level 2:

```
from dataclasses import dataclass
```

```
from typing import Generic, TypeVar
```

```
T = TypeVar("T")
```

```
@dataclass
```

```
class APIResponse(Generic[T]):
```

```
    status_code: int
```

```
data: T
```

```
error_message: str | None = None
```

Level 3:

```
from dataclasses import dataclass
```

```
class Comparable:
```

```
    def is_greater(self, other: "Comparable")  
    -> bool: ...
```

```
# T must be a subclass of Comparable
```

```
def find_max[T: Comparable](items:  
list[T]) -> T:
```

```
    max_item = items[0]
```

```
    for item in items[1:]:
```

```
        if item.is_greater(max_item):
```

```
            max_item = item
```

```
    return max_item
```

3. SCHEMA VALIDATORS

i. LOGGER AND TYPE

VARIABLE

ii. FIELD ERROR

iii. VALIDATION OUTCOME

iv.REJECT

v.VALIDATION PAYLOAD

vi.VALIDATION BATCH

4.CONNECTION

5.CONCLUSION